AWS システム設計資料 - 簡易な監視・ログ管理システム

システム概要

CloudWatch を活用し、アプリケーションとインフラのログを集約・分析・監視するシステム。リアルタイムアラートと可視化により、システム健全性を維持します。

対象用途:

- アプリケーション監視
- パフォーマンス分析
- トラブルシューティング
- コンプライアンス監査
- リアルタイムアラート

アーキテクチャ構成

```
アプリケーション層
├─ Lambda
├─ EC2
├─ API Gateway
├─ RDS
└─ DynamoDB
    ↓ ログ送信
CloudWatch Logs
├─ ロググループ (/aws/lambda/...)
├─ ロググループ (/aws/rds/...)
├─ ロググループ (/app/...)
└─ ロググループ (/api/...)
    ↓ ログ処理
CloudWatch Logs Insights
├─ クエリ実行
├─ 分析
└─ フィルタリング
    ↓
CloudWatch Metrics
├─ カスタムメトリクス
├─ AWS メトリクス
└─ メトリクスフィルター
    ↓
CloudWatch Alarms
├─ アラート条件
├─ SNS 通知
└─ Auto Scaling トリガー
    ↓
CloudWatch Dashboards
└─ 可視化・監視
```

使用するAWSサービス

サービス	役割	責務内容
CloudWatch Logs	ログ集約	複数ソースのログ一元管理
CloudWatch Metrics	メトリクス記録	システムメトリクス監視
CloudWatch Alarms	アラート	閾値ベースの通知
CloudWatch Dashboards	可視化	リアルタイムダッシュボード
SNS	通知配信	メール、SMS、Slack 通知
Lambda	ログ処理	カスタムログ処理関数

詳細設定値

CloudWatch ロググループ設定

```
ロググループ1: /aws/lambda/production
  - ロググループ名: /aws/lambda/production
  - リージョン: ap-northeast-1
  - ログ保持期間: 30日
  - KMS 暗号化: 無効（必要に応じ有効化）

ロググループ2: /aws/rds/database-logs
  - ロググループ名: /aws/rds/database-logs
  - ログ保持期間: 7日

ロググループ3: /application/api-gateway
  - ロググループ名: /application/api-gateway
  - ログ保持期間: 14日
  - メトリクスフィルター: 有効

ロググループ4: /application/business-logic
  - ロググループ名: /application/business-logic
  - ログ保持期間: 90日（重要ログ）

ログストリーム命名規則:
  - Lambda: instance-id-{instance-id}
  - 例: instance-id-12345678

ログフォーマット例 (JSON):
  {
    "timestamp": "2024-11-02T10:30:00.123Z",
    "level": "INFO",
    "requestId": "req-uuid-12345",
    "userId": "user-001",
    "action": "CreateUser",
    "message": "User created successfully",
    "duration": 125,
```

```
    "metadata": {
      "email": "user@example.com",
      "source": "api"
    }
  }

ロググループ設定：

ログフィルター設定:

  1. フィルター名: ErrorMessages
     - パターン: [... , "level" = "ERROR", ...]
     - メトリクス名: ErrorCount
     - メトリクス値: 1
     - デフォルト値: 0

  2. フィルター名: SlowRequests
     - パターン: [... , duration > 3000, ...]
     - メトリクス名: SlowRequestCount
     - メトリクス値: 1
     - デフォルト値: 0

  3. フィルター名: FailedOperations
     - パターン: [... , "status" = "failed", ...]
     - メトリクス名: FailedOperationCount
     - メトリクス値: 1
```

CloudWatch メトリクス設定

```
メトリクス名前空間: ApplicationMetrics

カスタムメトリクス1: RequestLatency
  - 単位: ミリ秒
  - 統計値: Average, Maximum, Minimum, p50, p90, p99
  - ディメンション: Endpoint, Method, StatusCode
  - 集計値間隔: 1分

カスタムメトリクス2: UserActiveSessions
  - 単位: Count
  - 統計値: Average, Maximum
  - ディメンション: ApplicationVersion
  - 集計値間隔: 5分

カスタムメトリクス3: DatabaseConnectionPool
  - 単位: Count
  - 統計値: Average, Maximum
  - ディメンション: DatabaseInstance
  - 集計値間隔: 1分

カスタムメトリクス4: ApiCallCount
  - 単位: Count/s
```

```
  - 統計値: Sum
  - ディメンション: ApiVersion, ClientType
  - 集計値間隔: 1分

複合メトリクス:
  - メトリクス式: ErrorRate = (ErrorCount / TotalRequests) * 100
  - 表示: ダッシュボードに表示
```

CloudWatch アラーム設定

```
アラーム1: HighErrorRate
  - 名前: HighErrorRate
  - メトリクス: ErrorCount
  - 統計: Sum
  - 期間: 5分
  - 評価期間: 2回連続
  - 閾値: 100件/5分
  - 比較演算子: >= (以上)
  - アラーム状態: アラーム → SNS 通知
  - 信頼度: Medium（Low に設定して誤検知減）

アラーム2: HighLatency
  - メトリクス: RequestLatency
  - 統計: p99
  - 期間: 5分
  - 評価期間: 1回
  - 閾値: 5000ms
  - 比較演算子: >
  - アクション:
    * SNS: alert-topic-arn
    * Email: ops-team@example.com

アラーム3: DatabaseCpuUtilization
  - メトリクス: DBCPUUtilization
  - 統計: Average
  - 期間: 1分
  - 評価期間: 2回連続
  - 閾値: 80%
  - 比較演算子: >
  - アクション:
    * SNS トピック
    * Auto Scaling ポリシー

アラーム4: LowDiskSpace
  - メトリクス: DiskSpaceUtilization
  - 統計: Average
  - 期間: 5分
  - 評価期間: 1回
  - 閾値: 90%
  - 比較演算子: >
  - アクション: PagerDuty 連携
```

```
アラーム5: ApiGatewayThrottling
  - メトリクス: ApiGateway Throttle Count
  - 統計: Sum
  - 期間: 1分
  - 評価期間: 1回
  - 閾値: 0件（任意のスロットル）
  - 比較演算子: >
  - アクション: 即座に通知

異常検知アラーム:
  - メトリクス: RequestCount
  - 異常検知モデル: 適応型（過去2週間学習）
  - 感度: 2標準偏差
  - データポイント評価: 直近2週間のパターンから異常判定

複合アラーム:
  - 名前: CriticalSystemFailure
  - 条件: HighErrorRate AND HighLatency OR DatabaseDown
  - アクション: 最高優先度で通知
```

CloudWatch ダッシュボード設定

```
ダッシュボード名: Production-Monitoring-Dashboard

レイアウト: 4列 × 複数行

セクション1: リアルタイムメトリクス

  ウィジェット1.1: RequestsPerSecond
    - タイプ: 折れ線グラフ
    - メトリクス: RequestCount (Sum)
    - 期間: 1分
    - 期間表示: 過去1時間

  ウィジェット1.2: ErrorRate
    - タイプ: 折れ線グラフ
    - メトリクス: ErrorCount / RequestCount * 100
    - 期間: 1分
    - 警告: 5%以上は赤表示

  ウィジェット1.3: Latency (p99)
    - タイプ: 折れ線グラフ
    - メトリクス: RequestLatency (p99)
    - 期間: 1分
    - 警告: 5000ms以上は赤表示

  ウィジェット1.4: Active Users
    - タイプ: 数値表示
    - メトリクス: ActiveSessions
    - 期間: 1分
```

```
セクション2: システムリソース

  ウィジェット2.1: CPU Utilization
    - タイプ: 棒グラフ
    - メトリクス: CPUUtilization
    - ディメンション: InstanceId
    - 期間: 5分

  ウィジェット2.2: Memory Usage
    - タイプ: 折れ線グラフ
    - メトリクス: MemoryUtilization
    - 期間: 5分

  ウィジェット2.3: Disk Space
    - タイプ: ゲージ
    - メトリクス: DiskUtilization
    - 期間: 5分

  ウィジェット2.4: Network I/O
    - タイプ: 折れ線グラフ
    - メトリクス: NetworkIn, NetworkOut
    - 期間: 1分

セクション3: アプリケーションメトリクス

  ウィジェット3.1: Lambda Duration
    - タイプ: 折れ線グラフ
    - メトリクス: Duration (Average, Max)
    - ディメンション: FunctionName
    - 期間: 1分

  ウィジェット3.2: Lambda Errors
    - タイプ: 棒グラフ
    - メトリクス: Errors (Sum)
    - ディメンション: FunctionName
    - 期間: 1分

  ウィジェット3.3: DynamoDB Consumed Capacity
    - タイプ: 積み上げ棒グラフ
    - メトリクス: ConsumedReadCapacity, ConsumedWriteCapacity
    - 期間: 1分

  ウィジェット3.4: RDS Connections
    - タイプ: 折れ線グラフ
    - メトリクス: DatabaseConnections
    - 期間: 1分

セクション4: ログインサイト

  ウィジェット4.1: Recent Errors
    - タイプ: ログインサイト
    - ロググループ: /aws/lambda/production
    - クエリ: fields @timestamp, @message, level
```

```
                | filter level = "ERROR"
                | stats count() by @message
    - 期間: 過去1時間

  ウィジェット4.2: Slow Queries
    - タイプ: テーブル表示
    - ロググループ: /application/business-logic
    - クエリ: fields @timestamp, duration, query
                | filter duration > 3000
                | stats avg(duration), max(duration) by query
    - 期間: 過去30分

  ウィジェット4.3: API Response Status Distribution
    - タイプ: 円グラフ
    - ロググループ: /application/api-gateway
    - クエリ: fields status
                | stats count() by status
    - 期間: 過去1時間
```

SNS トピック設定

```
SNS トピック1: alert-topic-critical
  - トピック名: arn:aws:sns:ap-northeast-1:ACCOUNT:alert-topic-critical
  - 説明: 重大なアラート

  サブスクリプション:
    1. メール: ops-critical@example.com
    2. SMS: +81-90-xxxx-xxxx
    3. PagerDuty: PagerDuty Integration
    4. Slack: Webhook URL

SNS トピック2: alert-topic-warning
  - トピック名: arn:aws:sns:ap-northeast-1:ACCOUNT:alert-topic-warning
  - 説明: 警告レベルアラート

  サブスクリプション:
    1. メール: ops-team@example.com
    2. Slack: Webhook URL
    3. Lambda: alert-processor

SNS トピック3: alert-topic-info
  - トピック名: arn:aws:sns:ap-northeast-1:ACCOUNT:alert-topic-info
  - 説明: 情報通知

  サブスクリプション:
    1. メール: logs-archive@example.com (フィルター: Digests only)
    2. S3: ログアーカイブバケット
```

CloudWatch Logs Insights クエリ例

```
クエリ1: エラー率の時系列

fields @timestamp, status
| stats count() as total_requests,
        sum(case when status >= 400 then 1 else 0 end) as errors by bin(5m)
| stats (errors / total_requests) * 100 as error_rate by bin(5m)


クエリ2: スローリクエスト分析

fields @timestamp, @duration, @message, userId
| filter @duration > 3000
| stats count() as slow_count,
        avg(@duration),
        max(@duration),
        pct(@duration, 99) by userId
| sort slow_count desc


クエリ3: ユーザーあたりのリクエスト集計

fields @timestamp, userId, action
| stats count() as action_count by userId, action
| sort action_count desc


クエリ4: 時間帯別トレンド

fields @timestamp
| stats count() as request_count by bin(1h)
| sort @timestamp desc


クエリ5: ステータスコード別分布

fields status
| stats count() by status
| sort count desc


クエリ6: Lambda 関数ごとのエラー率

fields @duration, @message, @logStream
| filter @message like /ERROR/
| stats count() as error_count,
        avg(@duration) as avg_duration by @logStream
| sort error_count desc


クエリ7: 特定ユーザーの活動追跡

fields @timestamp, @message, action
```

```
| filter userId = "user-001"
| stats count() by action, bin(1h)
```

推定月額コスト

シナリオ: 月間ログ100GB、1000メトリクス、20アラーム

項目	使用量	単価	月額費用
CloudWatch Logs 取得	100 GB	$0.50/GB	$50.00
CloudWatch Logs ストレージ	30日分	$0.03/GB/月	$90.00
CloudWatch Logs Insights	1000クエリ	$0.005/クエリ	$5.00
CloudWatch メトリクス	1000個	$0.30/メトリクス	$300.00
CloudWatch アラーム	20個	$0.10/アラーム	$2.00
CloudWatch ダッシュボード	5個	$3.00/ダッシュボード	$15.00
SNS メール通知	1000通	無料	$0.00
合計			**$462.00**

デプロイメント手順

1. ロググループ作成

```
aws logs create-log-group \
  --log-group-name /application/api-gateway \
  --region ap-northeast-1

aws logs put-retention-policy \
  --log-group-name /application/api-gateway \
  --retention-in-days 14 \
  --region ap-northeast-1
```

2. メトリクスフィルター設定

```
aws logs put-metric-filter \
  --log-group-name /application/api-gateway \
  --filter-name ErrorMetricsFilter \
  --filter-pattern "[... , level = ERROR, ...]" \
  --metric-transformations \
    metricName=ErrorCount,metricNamespace=ApplicationMetrics,metricValue=1 \
  --region ap-northeast-1
```

3. SNS トピック作成

```
aws sns create-topic \
  --name alert-topic-critical \
  --region ap-northeast-1

aws sns subscribe \
  --topic-arn arn:aws:sns:ap-northeast-1:ACCOUNT:alert-topic-critical \
  --protocol email \
  --notification-endpoint ops-critical@example.com
```

4. アラーム作成

```
aws cloudwatch put-metric-alarm \
  --alarm-name HighErrorRate \
  --alarm-description "Alert when error count exceeds 100" \
  --metric-name ErrorCount \
  --namespace ApplicationMetrics \
  --statistic Sum \
  --period 300 \
  --threshold 100 \
  --comparison-operator GreaterThanOrEqualToThreshold \
  --evaluation-periods 2 \
  --alarm-actions arn:aws:sns:ap-northeast-1:ACCOUNT:alert-topic-critical \
  --region ap-northeast-1
```

監視とログ分析のベストプラクティス

1. ログレベルの設定

```
DEBUG:   詳細情報（開発環境のみ）
INFO:    処理結果、重要なイベント
WARNING: 潜在的な問題
ERROR:   エラーが発生したが処理継続
FATAL:   システム停止
```

2. ログ構造化推奨事項

```
- JSON 形式を使用
- タイムスタンプを含める
- リクエストID でトレース可能に
- ユーザーID を記録
- エラースタックトレース含める
```

3. 保持期間ガイドライン

```
- 本番ログ: 30日
- セキュリティログ: 90日
- 監査ログ: 1年
- アーカイブ: S3 Glacier で保存
```